

CO2_AT1_Database Design and Connectivity

DSA05 – Query Processing for Data Science

S. Sharon Surya

192324246

Q1 Library Book Management System (10 Marks)

A library plans to automate the management of its books using PostgreSQL. Design a Book table containing Book ID, Title, Author, Publisher, Year of Publication, and Price. Write a Python program to establish a connection with the database, create the table, insert sample records, update the price of a selected book, and display the updated information.

Program:

```
import psycopg2

# Connect to PostgreSQL
conn = psycopg2.connect(
    host="localhost",
    database="librarydb",
    user="postgres",
    password="sergtsop",
    port="5432"
)

cur = conn.cursor()

# Create Book table
cur.execute("""
CREATE TABLE IF NOT EXISTS Book(
    BookID INT PRIMARY KEY,
    Title VARCHAR(100),
    Author VARCHAR(100),
    Publisher VARCHAR(100),
    YearOfPublication INT,
    Price NUMERIC(8,2)
)
""")

# Insert sample records
cur.execute("""
INSERT INTO Book VALUES
(101,'Python Programming','Mark Lutz','OReilly',2020,650.00),
```

```
(102,'Database Systems','Navathe','Pearson',2019,750.00),
(103,'Data Mining','Han','Elsevier',2021,900.00)
ON CONFLICT (BookID) DO NOTHING;
""")
```

```
# Update price of BookID 103
cur.execute("""
UPDATE Book
SET Price = 950.00
WHERE BookID = 103;
""")
```

```
conn.commit()
```

```
# Display updated records
cur.execute("SELECT * FROM Book")
```

```
rows = cur.fetchall()
```

```
print("\nUpdated Book Details")
print("-" * 80)
```

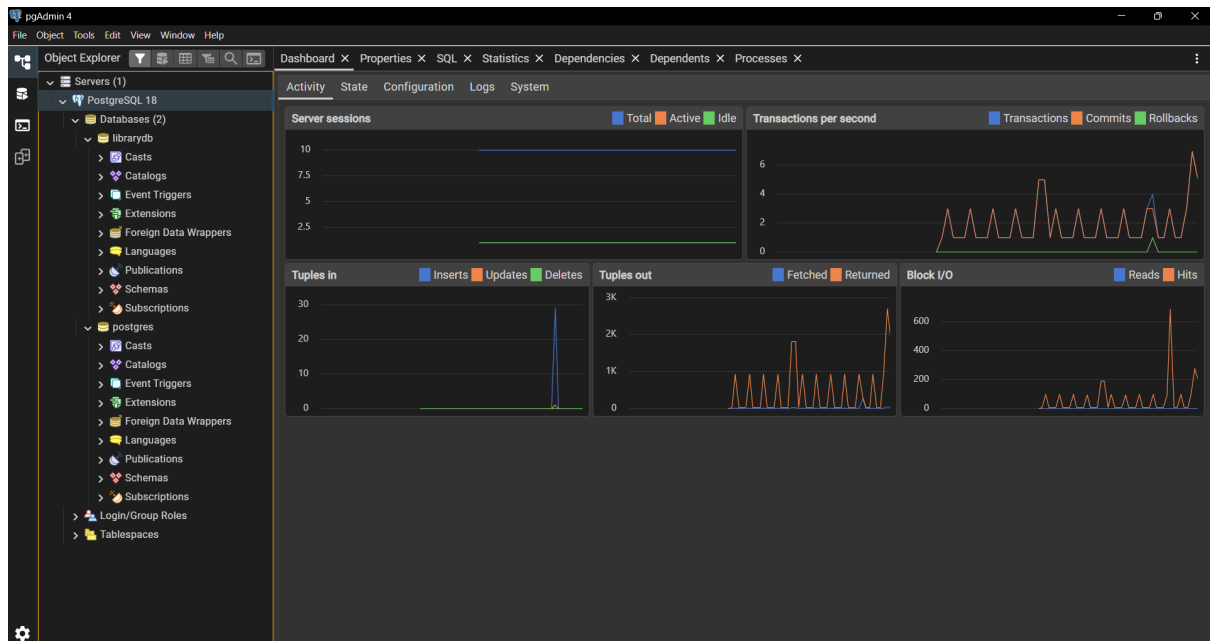
```
for row in rows:
    print(row)
```

```
cur.close()
conn.close()
```

Output:

Updated Book Details

```
-----
(101, 'Python Programming', 'Mark Lutz', 'OReilly', 2020, Decimal('650.00'))
(102, 'Database Systems', 'Navathe', 'Pearson', 2019, Decimal('750.00'))
(103, 'Data Mining', 'Han', 'Elsevier', 2021, Decimal('950.00'))
```



The screenshot shows a Python IDE with a file named 'library.py'. The code defines a connection to a PostgreSQL database and inserts data into a table named 'books'.

```

1 import psycopg2
2
3 # Connect to PostgreSQL
4 conn = psycopg2.connect(
5     host="localhost",
6     database="librarydb",
7     user="postgres",
8     password="sangtson",
9     port="5432"
10 )
11
12 cur = conn.cursor()
13
14 # Create Book table

```

The 'Run' output shows the following data being inserted:

```

Updated Book Details
-----
(101, 'Python Programming', 'Mark Lutz', 'O'Reilly', 2020, Decimal('650.00'))
(102, 'Database Systems', 'Navathe', 'Pearson', 2019, Decimal('750.00'))
(103, 'Data Mining', 'Han', 'Elsevier', 2021, Decimal('950.00'))

```

Process finished with exit code 0

Q2. Examination Result Management System (10 Marks)

A college wants to store student details and examination results in an SQLite database. Design Student and Result tables with appropriate relationships. Write a Python program to connect to the SQLite database, create the tables, insert sample records, calculate the total marks for each student using SQL queries, and display the student names along with their total marks.

Program:

```
import sqlite3

# Connect to SQLite database
conn = sqlite3.connect("college.db")

cur = conn.cursor()

# Create Student table
cur.execute("""
CREATE TABLE IF NOT EXISTS Student(
    StudentID INTEGER PRIMARY KEY,
    StudentName TEXT
)
""")

# Create Result table
cur.execute("""
CREATE TABLE IF NOT EXISTS Result(
    StudentID INTEGER,
    Subject1 INTEGER,
    Subject2 INTEGER,
    Subject3 INTEGER,
    FOREIGN KEY(StudentID) REFERENCES Student(StudentID)
)
""")

# Insert sample students
cur.execute("INSERT OR IGNORE INTO Student VALUES (1,'Rahul')")
cur.execute("INSERT OR IGNORE INTO Student VALUES (2,'Priya')")
cur.execute("INSERT OR IGNORE INTO Student VALUES (3,'Arun')")

# Insert sample results
cur.execute("INSERT OR IGNORE INTO Result VALUES (1,85,90,88)")
cur.execute("INSERT OR IGNORE INTO Result VALUES (2,80,84,86)")
cur.execute("INSERT OR IGNORE INTO Result VALUES (3,91,93,95)")

conn.commit()

# Calculate total marks
cur.execute("""
SELECT Student.StudentName,
(Result.Subject1 + Result.Subject2 + Result.Subject3) AS TotalMarks
FROM Student
JOIN Result
ON Student.StudentID = Result.StudentID
""")

rows = cur.fetchall()
```

```
print("Student Results")
print("-" * 30)

for row in rows:
    print(f"Name: {row[0]} Total Marks: {row[1]}")

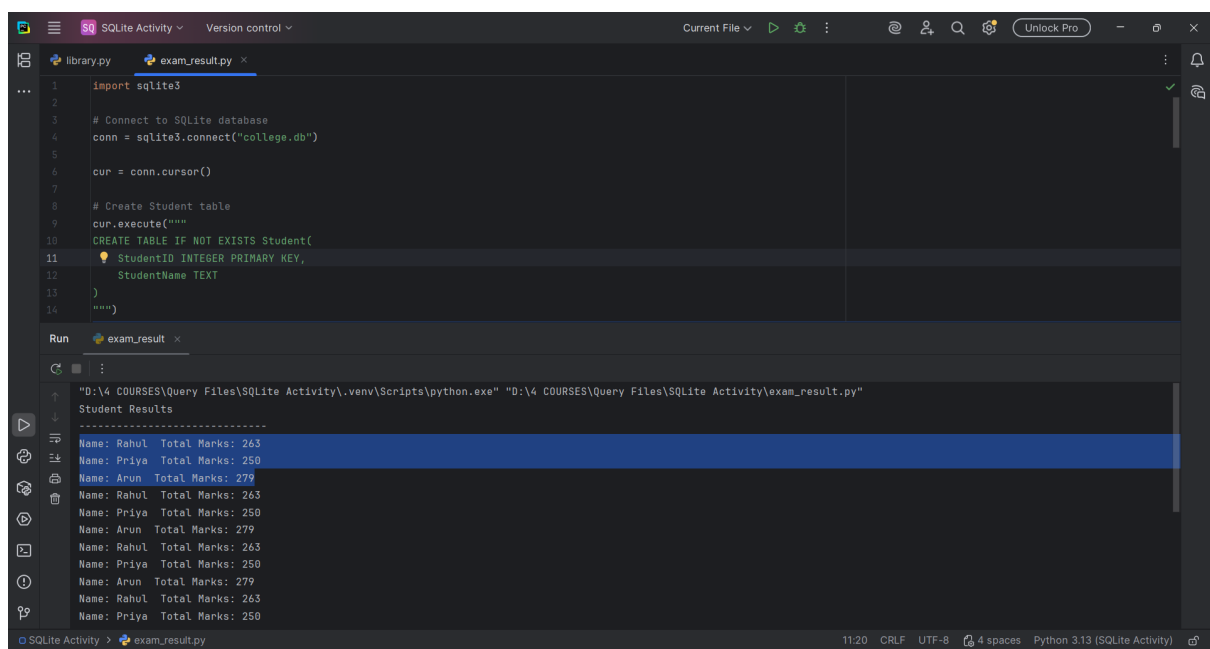
conn.close()
```

Output:

Name: Rahul Total Marks: 263

Name: Priya Total Marks: 250

Name: Arun Total Marks: 279



The screenshot shows a code editor with a Python script named `exam_result.py` and its output in the Run console.

Code:

```
1 import sqlite3
2
3 # Connect to SQLite database
4 conn = sqlite3.connect("college.db")
5
6 cur = conn.cursor()
7
8 # Create Student table
9 cur.execute("""
10 CREATE TABLE IF NOT EXISTS Student(
11     StudentID INTEGER PRIMARY KEY,
12     StudentName TEXT
13 )
14 """)
```

Run Output:

```
"D:\4 COURSES\Query Files\SQLite Activity\.venv\Scripts\python.exe" "D:\4 COURSES\Query Files\SQLite Activity\exam_result.py"
Student Results
-----
Name: Rahul Total Marks: 263
Name: Priya Total Marks: 250
Name: Arun Total Marks: 279
Name: Rahul Total Marks: 263
Name: Priya Total Marks: 250
Name: Arun Total Marks: 279
Name: Rahul Total Marks: 263
Name: Priya Total Marks: 250
Name: Arun Total Marks: 279
Name: Rahul Total Marks: 263
Name: Priya Total Marks: 250
```

